

PLAYS: Minimizing DNN Inference Latency in Serverless Edge Cloud for Artificial Intelligence of Things

Hongmin Geng, Deze Zeng[✉], Senior Member, IEEE, Yuepeng Li[✉], Lin Gu[✉], Member, IEEE, Quan Chen[✉], Member, IEEE, and Peng Li[✉], Senior Member, IEEE

Abstract—Thanks to the capability of fine-grained resource allocation and fast task scheduling, serverless computing has been adopted into edge cloud to accommodate various applications, e.g., deep neural network (DNN) inference for Artificial Intelligence of Things (AIoT). In serverless edge cloud, the servers are started up on-demand. However, as a container-based architecture, the inherent sequential startup feature of container imposes high affection on the DNN inference performance in serverless edge clouds. In this article, we investigate the distributed DNN inference problem in serverless edge cloud with the consideration of such characteristics, aiming to eliminate the extra container startup time cost to minimize the DNN inference latency. We formulate this problem into a nonlinear optimization form and then linearize it into an integer programming problem, which is proved as NP-hard. To tackle the computation complexity, we propose a priority-based layer scheduling (PLAYS) algorithm. Extensive experiment results verify the effectiveness and the adaptability of our PLAYS algorithm in comparison with other state-of-art algorithms under several well known DNN models.

Index Terms—Artificial Intelligence of Things (AIoT), distributed Deep neural network (DNN) inference, serverless edge cloud, task scheduling.

I. INTRODUCTION

THE LAST decade has witnessed the unprecedented progress of the artificial intelligence (AI) in Internet of

Things (IoT) toward the vision of Artificial Intelligence of Things (AIoT). AIoT aims to enhance various applications by combining intelligent algorithms with a vast array of physical devices, enabling smarter and automated environments for real-time monitoring, management, and data analysis.

Deep neural networks (DNNs), serving as the cornerstone technology of AIoT, play a crucial role in diverse domains. For instance, AIoT-based air quality monitoring systems [1] enhance environmental management in urban areas by applying DNN technology. Moreover, DNN contributes to AIoT city surveillance system [2], improving security monitoring of public spaces. In addition, DNN drive advancements in AIoT-based autonomous driving and vehicular network [3], leading the way for safer and efficient urban mobility.

Albeit with high potential, executing DNN models directly on those AIoT devices is often impractical due to the computational power constraints. Consequently, offloading DNN inference tasks to edge clouds has emerged as an effective solution. Meanwhile, the varying workload and computing power requirements of such tasks necessitate efficient and elastic resource management at the edge cloud.

Within edge clouds, serverless computing has garnered significant attention as an emerging computational paradigm, owing to its on-demand provisioning of computational resources. This paradigm enables AIoT applications to dynamically adjust resource allocation based on task demands, thereby achieving higher resource utilization and cost-effectiveness [4]. Nevertheless, since the edge cloud has been widely regarded as resource-constrained, a single edge server is not invariably sufficient to satisfy the resource demands associated with DNN inference tasks, which often require substantial computational power and memory. To tackle such problems, distributed DNN inference has emerged as an appealing solution by allowing the edge cloud to execute part of the DNN inference task [5].

A DNN inference task executing in the serverless edge cloud is usually decomposed into a set of dependent functions (e.g., a set of DNN layers as a function), which are encapsulated into containers. Once the function is invoked, the corresponding container will be started up on the specified server for execution [6], [7], [8], [9]. We first make preliminary experiments to understand how the startup time affects the overall performance. We start up a set of different containers with different DNN models and record their startup time and execution time. The results are reported in Fig. 1, from which

Manuscript received 20 May 2024; revised 17 July 2024; accepted 31 July 2024. Date of publication 14 August 2024; date of current version 20 November 2024. This work was supported in part by the Natural Science Foundation of China under Grant 62172375, Grant 62232011, and Grant 62372200; in part by the Provincial Key Research and Development Program of Hubei under Grant 2023BAB065; in part by the CUG Scholar Scientific Research Funds at China University of Geosciences (Wuhan) under Grant 2022179; in part by the Japan Society for the Promotion of Science (JSPS) KAKENHI under Grant 24K02932; and in part by the Japan Science and Technology Agency (JST) PRESTO under Grant 23828673. (Corresponding author: Deze Zeng.)

Hongmin Geng and Deze Zeng are with the Engineering Research Center of Natural Resource Information Management and Digital Twin Engineering Software, Ministry of Education, and the School of Computer Science, China University of Geosciences, Wuhan 430074, China (e-mail: hmgeng@cug.edu.cn; deze@cug.edu.cn).

Yuepeng Li and Quan Chen are with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: chen-quan@sjtu.edu.cn).

Lin Gu is with the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: lingui@hust.edu.cn).

Peng Li is with the School of Computer Science and Engineering, University of Aizu, Aizuwakamatsu 965-8580, Japan (e-mail: pengli@u-aizu.ac.jp).

Digital Object Identifier 10.1109/IIOT.2024.3443289

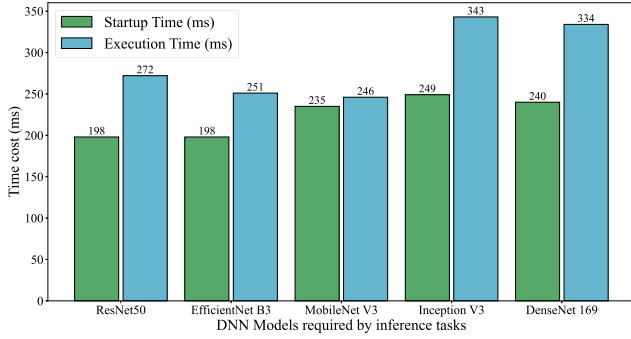


Fig. 1. Startup time and execution time of serverless functions under different DNN model inference tasks.

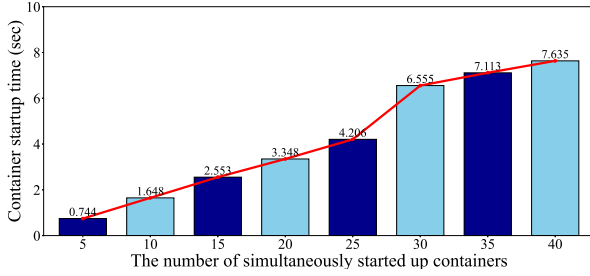


Fig. 2. Completion time of starting up different numbers of containers. It can be observed that the container startup time shows almost a linearly positive correlation with the number of containers.

we observe that during the lifecycle of a function, the startup time of a container may account as high as 40%–50%. This indicates that the startup time is a nonnegligible issue as it seriously affects the efficiency of inference services in a serverless edge cloud environment.

Furthermore, we find another critical characteristic of serverless computing that is usually ignored. That is, the containers on the same server must be started up sequentially, which is corroborated in recent studies [10], [11]. To gain a deeper insight, we also conduct a preliminary experiment to evaluate the average startup time by increasing the number of simultaneously started-up containers, as illustrated in Fig. 2. This is because, to guarantee the isolation of container, the operation system must first create a network namespace for each container. However, the creation of the network namespace need to monopolize the shared configuration file for the content consistency consideration. The corresponding locking operations make the containers on the same server be started up one by one, resulting in a lengthy startup time.

This characteristic contradicts the principle of conventional distributed DNN inference algorithms, i.e., scheduling layers onto a limited number of powerful servers to reduce the data exchange overhead, and leveraging sufficient computational resources. As a result of such sequential startup, the computing power-preferred scheduling method will inevitably lead to significant waiting times for containers, resulting in poor performance. Conversely, the load balancing-preferred scheduling method reduces the container startup time, but

leads to increased data exchange and prolonged DNN layer execution time.

Motivated by such fact, in this article, we investigate the distributed DNN inference problem in serverless edge cloud, aiming at minimizing the DNN inference time, which is critical to real-time AIoT applications. It is desirable, but challenged, to find a solution that can well balance the layer execution time, data exchanging overhead, and the container startup time due to the heterogeneous characteristic of serverless edge cloud. This article proposes an efficient solution called priority-based layer scheduling (PLAYS) and mainly makes the following contributions.

- 1) To our best knowledge, we are the first to study the distributed DNN inference problem for inference time minimization in a serverless edge cloud, with special consideration of the container's sequential startup feature. The problem is formulated as a nonlinear integer programming problem and proved to be NP-hard.
- 2) Due to the NP-hardness of the problem, we propose a PLAYS algorithm in polynomial computation complexity. The achievable performance is also theoretically analyzed.
- 3) To evaluate the effectiveness and adaptability of PLAYS, we conduct extensive experiments on several well-known DNN models. The experimental results show that our algorithm can reduce the inference time by 28%–40% averagely in comparison with other state-of-the-art methods.

The remainder of this article is organized as follows. In Section II a brief review on the related works is discussed. Section III states the system model and problem formulation. Section IV presents our proposed PLAYS algorithm in detail and give the theoretical analysis for the performance. The performance evaluation of our proposed algorithm is analyzed in Section V, and finally Section VI concludes this article.

II. RELATED WORKS

The edge cloud has become increasingly important for AIoT applications. As a result, existing studies focus on executing the DNN-based application by collaborating both edge cloud and IoT device [12], [13]. To fully utilized the computing resource, Zeng et al. [12] designed an on-demand DNN inference cooperative framework for AIoT. The inference task is adaptively partitioned and assigned to both IoT devices and edge servers. Chen et al. [13] achieved an automatic and dynamical DNN inference task offloading mechanism for edge servers.

Previous research has yielded certain results, but the way tasks are divided for collaborative processing is too broad, leading to unnecessary use of computing resources. To maximize the use of edge cloud for AIoT applications, distributed DNN inference has emerged as a key area of interest. As a result, many studies have introduced various algorithms aimed at reducing the latency of DNN inference. For example, Mohammed et al. [14] proposed a distributed inference acceleration algorithm, which partitions the DNN model and schedules the partitions to the appropriate server

TABLE I
KEY NOTATIONS

Notations	Definitions
$\mathbb{N}$	The set of edge servers
C_n	The computational capacity of edge server n
B_{mn}	The network bandwidth between servers m and n
$\mathbb{G}$	The graph of DNN inference task
$\mathbb{L}$	The vertexes set of DNN layers
$\mathbb{E}$	The edges set of dependency relationship between DNN layers
W_l	The workload of DNN layer l
C_l	The computational resources requirement of DNN layer l
x_{ln}	The binary variables indicates whether DNN layer l is scheduled to edge server n
$T_c(n)$	The time cost of container startup on edge server n
$T_c^s(l)$	The container startup time for DNN layer l
$T_c^f(l)$	The finish time of container startup for DNN layer l
$T_f(l)$	The inference finish time of DNN layer l

based on a matching game approach. A distributed DNN computing system, CoEdge [15], dynamically partitions the DNN inference task according to the devices' computing and network resources. Moreover, to exert the potential power of edge computing system in low delay, Wu et al. [16] introduced an innovative framework and constructs a finer grained offloading strategy predicated on the partitioning of applications.

Albeit lots of algorithms have been proposed for inference acceleration, efficient resource management and task scheduling still remain the challenges that stymie the development of edge intelligence. To this end, serverless computing has been regarded as a potential technique to tackle these challenges [17]. Researchers have already studied how to leverage the serverless computing to assist the distributed DNN inference. For example, a GPU-supported serverless computing framework is proposed for inference accelerating [18]. Yu et al. [9] proposed a serverless-based model serving system, Gillis, which automatically partitions a large DNN model across multiple serverless functions for inference acceleration and memory usage reduction.

These studies suggest that serverless computing is a potential computing paradigm to distributed DNN inference to better utilize the resources in either central cloud and edge cloud. However, we observe that all these studies fail to consider an important characteristic of the container, which is the key enabling technology of serverless computing. As discussed above, the container must be started up sequentially. The accumulated startup time makes the DNN inference tasks suffer severe performance degradation in serverless edge cloud. To address this gap, we are motivated to study how to accelerate the distributed DNN inference with the consideration sequential startup overhead in this article. Our work is orthogonal to existing serverless computing empowered distributed DNN inference as it mainly focuses on the underlying technology optimization.

III. SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we introduce the system model and give the formulation of the optimization problem studied in this article.

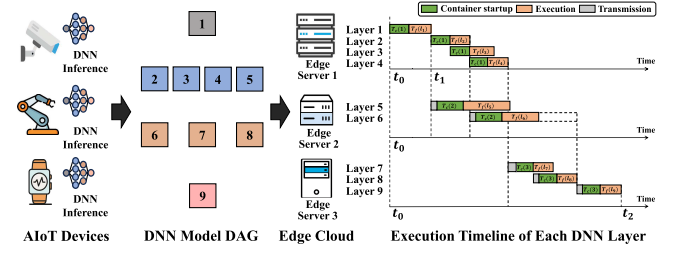


Fig. 3. DNN inference task execution process in serverless edge cloud.

A. System Model

We consider a serverless edge cloud and the DNN inference task are offloaded to it on a layerwise manner for execution. The key notations in this work are described in Table I. Following the works in [19], the serverless edge cloud comprises a set $\mathbb{N}$ of edge servers. For each edge server $n \in \mathbb{N}$, the computational capacity is C_n . Any two edge servers can communicate with each other, either directly or indirectly. Without loss of generality, we use B_{mn} to denote the network bandwidth between servers m and n . Suppose there is a DNN inference task, represented as a DAG $\mathbb{G} = \langle \mathbb{L}, \mathbb{E} \rangle$, where $\mathbb{L}$ is a set of vertexes denoting the DNN layers, and $\mathbb{E}$ is a set of edges representing the dependency relationship of the DNN layers. An edge $e_{l,l'} \in \mathbb{E}$ indicates that l should be processed before l' , and the output of l is the input of l' . We use $d_{l,l'}$ to denote the data size transferred from layer l to l' . A DNN layer $l \in \mathbb{L}$ is indivisible in serverless edge cloud, and it will be encapsulated as a containerized serverless function for execution. When a serverless function is invoked, the corresponding container will be started up on a edge server $n \in \mathbb{N}$ first and startup time is denoted as $T_c(n)$. This variable indicates that the time cost for container startup can vary among different edge servers since the disparities in hardware capabilities, such as IO bandwidth. In addition, thanks to the fixed structure of DNN model, the workload of each DNN layer can be estimated, and indicated as $W(l)$. Actually, the computational resources required by the serverless function are usually set in advance (e.g., in `cgroup` configuration file for Docker container), and we denote the computational resources requirement of DNN layer l as $C(l)$.

A detailed explanation for execution process of DNN inference task in serverless edge cloud is illustrated in Fig. 3. The example DNN model consists of 9 layers, which exist parallel branch structures. For each layer, we assume once the required input data is ready, container is then started up to sustain the serverless function. Next, it executes the computation task and transmits the output results to its subsequent layer. As shown in execution timeline, after layer 1 finished its execution at time t_1 , layers 2–5 should have gotten started simultaneously, but they have to queue up for container startup. Due to the limited computational resource capacity, layer 5 is scheduled to edge server 2 and it costs extra transmission time for the required data from edge server 1, as similar as layers 6–9. Besides, the container startup time on each server (e.g., $T_c(1)$, $T_c(2)$ and $T_c(3)$) shows the difference, since the heterogeneous of edge

cloud. Finally, when layer 9 finished its execution at time t_2 , the inference task of example DNN model gets completed.

In next section, we will give a more detailed analysis for problem formulation and describe our optimization goal.

B. Problem Formulation

Since the DNN layers are indivisible, for any DNN layer $l \in \mathbb{L}$, it can only be executed on a single server. Binary variables $x_{ln} = \{0, 1\}$ are introduced to indicate whether the DNN layer l is scheduled to edge server n (i.e., $x_{ln} = 1$) or not (i.e., $x_{ln} = 0$). To ensure that the DNN inference can be completed, all the DNN layers must be scheduled to the edge servers. That is

$$\sum_{n \in \mathbb{N}} x_{ln} = 1 \quad \forall l \in \mathbb{L}. \quad (1)$$

For any DNN layer l , when all its predecessors have been completed, and all the required data have been received, the corresponding containerized serverless function will be invoked and the container should be started up. Hence, we define the startup time of container for DNN layer l as $T_c^s(l)$ and it should satisfy

$$T_c^s(l) \geq T_f(l'') + \frac{x_{l''m} x_{ln} d_{l'',l}}{B_{m,n}} \quad \forall m, n \in \mathbb{N} \quad \forall l \in \mathbb{L}, l'' \in \text{Pred}(l) \quad (2)$$

where $\text{Pred}(l)$ indicates the predecessor set of layer l .

Therefore, the finish time of container startup for l can be calculated as $T_c^f(l) = T_c^s(l) + (\sum_{n \in \mathbb{N}} x_{ln} T_c(n))$. Note that, the container can only be started up sequentially since the configuration file will be locked when creating the network namespace. If there are multiple containers will be started up on the same server, it should satisfy the following constraint:

$$T_c^s(l) \geq T_c^f(l') \parallel T_c^s(l') \geq T_c^f(l) \quad \forall l, l' \in \mathbb{L}, l \neq l'. \quad (3)$$

More specifically, (3) is designed to formulate the sequential startup nature of containers on a serverless edge cloud. In the left part of (3), the inequality $T_c^s(l) \geq T_c^f(l')$ indicates that the startup time of container l must occur no earlier than the finish time of another layer l' startup. In the right part, vice versa, the inequality $T_c^s(l') \geq T_c^f(l)$ is a reciprocal condition ensuring that layer l must have completed its startup before l' can begin. This constraint is essential for preventing startup conflicts and ensuring sequential container startup on the same edge server.

After the container has been started up, the DNN layer l can be executed. However, due to the heterogeneity of edge servers, the same computational resource may have different computational power. According to [20], we use β_n to indicate the computational power achieved by one computational resource unit on edge server n . The finish time of DNN layer l can be calculated as

$$T_f(l) = T_c^f(l) + \sum_{n \in \mathbb{N}} \frac{x_{ln} W(l)}{\beta_n C(l)} \quad \forall l \in \mathbb{L}. \quad (4)$$

In order to ensure that all the DNN layers have enough computing resources for execution, the total computational

resource requirements of DNN layers assigned to the server n should not exceed its computational resource capacity. That is

$$\sum_{l \in \mathbb{L}} x_{ln} C(l) \leq C_n \quad \forall n \in \mathbb{N}. \quad (5)$$

For a DNN inference task, the completion time is exactly the same as the maximum layers finish time. By summing up the above, the serverless distributed DNN inference problem, which aims at minimizing the inference time by considering and mitigating the container startup overhead, can be formulated as

$$\begin{aligned} \min \quad & \{\max_{l \in \mathbb{L}} \{T_f(l)\}\} \\ \text{s.t.:} \quad & (1)-(3), (5). \end{aligned}$$

C. Complexity Analysis

Theorem 1: The serverless distributed DNN inference problem considering sequential container startup is NP-hard.

Proof: Considering a special case that the network bandwidth is infinite. And hence, the data transmission time can be ignored, and (2) can be converted to the following form:

$$T_c^s(l) \geq T_f(l'') \quad \forall l \in \mathbb{L}, l'' \in \text{Pred}(l). \quad (6)$$

Then, the original problem is transformed into a special case and the can be formulated as follows:

$$\begin{aligned} \min \quad & \{\max_{l \in \mathbb{L}} \{T_f(l)\}\} \\ \text{s.t.:} \quad & (1), (3), (5), (6). \end{aligned}$$

In this case, the serveless distributed DNN inference problem can be regarded as a typical flow shop scheduling problem. The flow shop scheduling problem conventionally involves a set of workers, denoted as W , and a set of jobs, denoted as J . Each job $j \in J$ consists of m distinct operations, each of which must be exclusively assigned to a single worker. It is mandatory that these operations are performed in a prescribed sequence for any given job. Nevertheless, the order in which jobs are executed within the set J can be adjusted as needed. The overarching goal in addressing the flow shop scheduling problem is to devise an optimal job allocation strategy that minimizes the maximum completion time across all jobs.

Similarly, the set of edge servers $\mathbb{N}$ can be viewed as the set of workers, denoted as W , while the DNN layers set $\mathbb{L}$ can be regarded as the set of jobs, denoted as J . Each DNN layer inference task $l \in \mathbb{L}$ represents an indivisible atomic operation, thus, considering operational constraints of jobs within the flow shop scheduling framework becomes unnecessary. Since the aim of this special case is to minimize the maximal finish time of DNN layer inference time through task scheduling, it inherently corresponds to the optimization objective of the flow shop scheduling problem, which has been proven to be NP-hard [21], [22]. Hence, the general case of this problem can also be classified as NP-hard. ■

IV. PRIORITY-BASED LAYER SCHEDULING ALGORITHM

Due to the NP-hardness of this problem, it is difficult to find the optimal solution in polynomial time. To tackle the computation complexity, existing studies often employ DAG-based scheduling algorithms [19], [23], [24]. However, these methods do not take into account the impact of container startup on the overall performance. To this end, we specifically propose a PLAYS algorithm in this section.

The key idea of PLAYS algorithm is scheduling the DNN layer to the appropriate edge servers in descending order of the priorities. In principle, we should guarantee the layer dependency when scheduling the DNN layers. That is, for a DNN layer l , its predecessors should be scheduled before it. In addition, to minimize the DNN inference time, for the DNN layers which can be executed in parallel, the DNN layer with more computational resources requirement should be scheduled earlier to enjoy the remained resources. Based on these considerations, we define the priority of the DNN layer l as follows:

$$\text{Priority}(l) = \max_{l' \in \text{succ}(l)} \{ \text{Priority}(l') + T(l) + T'_c(l) + T(l, l') \} \quad (7)$$

where $\text{succ}(l)$ denotes the successor set of DNN layer l . $T(l)$ and $T'_c(l)$ are the execution time and the container startup time of layer l on the server with highest computational power. $T(l, l')$ indicates the transmission time of data $d_{l,l'}$ on the network link with highest bandwidth.

Based on the priority of DNN layer, we propose PLAYS algorithm. The details of the algorithm are summarized in Algorithm 1. First, we calculate the priorities of all DNN layers in lines 1–3. Then, we select the layer with the highest priority and schedule it to the appropriate server (in lines 6–14). To find the appropriate edge server, we traverse all the available edge servers and calculate the finish time of the DNN layer on them (in lines 8–11). Note that, to satisfy the computational resource constraint, only the edge servers with enough residual resources will be traversed. Finally, the DNN layer will be scheduled to the server that can achieve the lowest finish time and accordingly we update the residual computational resource capacity of the edge server in lines 12–13. The above procedure repeats until all DNN layers are completely scheduled.

A. Theoretical Analysis

To show the efficiency of our proposed PLAYS algorithm, we theoretically analyze the performance gap between the inference time of PLAYS and optimal solution. Here, we denote them as T_p and T_o , respectively. For convenience, we discrete T_p into several time slots and categorize them into busy set $\mathbb{T}_b$ and idle set $\mathbb{T}_i$. $\mathbb{T}_b$ indicates the set of all time slots when all edge servers are busy and $\mathbb{T}_i$ includes the time slots when at least one edge server is idle. $\mathbb{T}_i$ can be represented by a disjoint union of k intervals, i.e., $\mathbb{T}_i = (b_{l_1}, b_{r_1}) \cup (b_{l_2}, b_{r_2}) \cup \dots \cup (b_{l_k}, b_{r_k})$, where $b_{l_1} < b_{r_1} < b_{l_2} < b_{r_2} < \dots < b_{l_k} < b_{r_k}$.

Algorithm 1 PLAYS Algorithm

```

1: for all  $l \in \mathbb{L}$  do
2:   Calculate the priority of  $l$  according to Eq. (7)
3: end for
4: while  $\mathbb{L} \neq \emptyset$  do
5:   Select the current layer  $l$  by the highest priority in  $\mathbb{L}$ 
6:   for all  $n \in \mathbb{N}$  do
7:      $\mathbb{T} \leftarrow \emptyset$ 
8:     if The remained computational resources of server  $n$ 
       is enough for layer  $l$  then
9:       Calculate finish time  $T_f(l)$  following Eq. (1)-(5)
10:       $\mathbb{T} \leftarrow \mathbb{T} \cup \{T_f(l)\}$ 
11:    end if
12:    Find the minimum  $T_f(l)$  from set  $\mathbb{T}$  and schedule
      layer  $l$  to the corresponding edge server
13:    Update the remained computational resources of the
      edge server
14:  end for
15:  Remove the current layer  $l$  and update the layer set by
       $\mathbb{L} \leftarrow \mathbb{L} \setminus l$ 
16: end while

```

Lemma 1: A chain of layers as $\mathbb{C} : c_p \rightarrow c_{p-1} \rightarrow \dots \rightarrow c_1$ can always be extracted from the DNN model, and it satisfies

$$\sum_{i=1}^k (b_{r_i} - b_{l_i}) \leq \sum_{j=1}^p \left(\frac{\beta_{\max}}{\beta_{\min}} T_{\min}(c_j) + T'_c(c_j) \right) + \sum_{p-1} T_{\max}^{\text{tran}} \quad (8)$$

where $T_{\max}^{\text{tran}}$ is the maximum data transmission time and $T_{\min}(l_j)$ is the minimum execution time of layer c_j . $\beta_{\max}$ and $\beta_{\min}$ denote the maximum and minimum computational power that per computation resource can provide, respectively.

Proof: Let c_1 denote the last layer scheduled by the PLAYS, and there are three cases correlated the start time of container startup for c_1 .

- 1) $T_c^s(c_1) \leq b_{l_1}$.
- 2) $T_c^s(c_1) \in \mathbb{T}_i$. That is, there exists an integer h , $h \leq k$, such that $b_{l_h} \leq T_c^s(c_1) \leq b_{r_h}$.
- 3) $T_c^s(c_1) \in \mathbb{T}_b$ but $T_c^s(c_1) > b_{l_1}$. That is, there exists an integer h , $h \leq k-1$, such that $b_{r_h} \leq T_c^s(c_1) \leq b_{l_{h+1}}$ or $b_{r_h} \leq T_c^s(c_1)$.

For the above three cases, we can always add one or more layers to the end of the chain to satisfy (8).

Case 1: In this case, layer c_1 itself can constitute a chain that satisfies (8).

Case 2: In this case, we can find another layer c_2 to construct chain $\mathbb{C}$ following two subcases.

SUB-CASE 1: If the container of c_1 can not be started up until $T_c^s(c_1)$, we simply add the last finished predecessor DNN layer into the chain.

SUB-CASE 2: If all the required data of c_1 have been received before $T_c^s(c_1)$, by the definition of $\mathbb{T}_i$, there must be an idle server n during (b_{l_h}, b_{r_h}) , and scheduling c_1 to n can achieve earlier container creation time. However, according to Algorithm 1, the reason why PLAYS does not schedule c_1 to n is that there must be another server n' satisfying $T_f^{n'}(c_1) \leq$

$T_f^n(c_1)$, where $T_f^n(c_1)$ and $T_f^{n'}(c_1)$ denote the finish time of c_1 scheduled to server n and n' , respectively. Let c_2 be the last finished predecessor of c_1 , and we have

$$T_f^n(c_1) \leq T_f(c_2) + T_{\max}^{\text{tran}} + T_{\max}(c_1) + T_c'(c_1) \quad (9)$$

where $T_{\max}(c_1)$ is the maximum execution time of layer c_1 , which can be calculated as $T_{\max}(c_1) = (\beta_{\max}/\beta_{\min})T_{\min}(c_1)$. Then, we can construct the chain via inserting layer c_2 into the end of c_1 and derive that

$$T_{\max}^{\text{tran}} + \frac{\beta_{\max}}{\beta_{\min}}T_{\min}(c_1) + T_c'(c_1) \geq T_f(c_1) - T_f(c_2). \quad (10)$$

Case 3: Same as *Case 2*, we insert the last finished immediate predecessor of c_1 into the chain, and obtain (10).

By adding layers in the way mentioned in *Case 2* or *Case 3* until *Case 1* happens, we can successfully construct chain $\mathbb{C}$ satisfying (8). ■

According to Lemma 1, we can derive the idle time in PLAYS as follows.

Lemma 2: The total idle time of all the servers in PLAYS satisfies

$$\sum_{n \in \mathbb{N}} T_{\text{idle}}(n) \leq |\mathbb{N}| \left(\frac{\beta_{\max}}{\beta_{\min}} T_o + \sum_{p=1} T_{\max}^{\text{tran}} \right) \quad (11)$$

where $T_{\text{idle}}(n)$ is the idle time of server n .

Proof: Obviously, for any server, the total idle time on it must be smaller than the total time in set $\mathbb{T}_i$, that is

$$\sum_{n \in \mathbb{N}} T_{\text{idle}}(n) \leq |\mathbb{N}| \sum_{i=1}^k (b_{r_i} - b_{l_i}). \quad (12)$$

With Lemma 1 and (12), we can derive

$$\sum_{n \in \mathbb{N}} T_{\text{idle}}(n) \leq |\mathbb{N}| \left(\sum_{j=1}^p \left(\frac{\beta_{\max}}{\beta_{\min}} T_{\min}(c_j) + T_c'(c_j) \right) + \sum_{p=1} T_{\max}^{\text{tran}} \right). \quad (13)$$

According to [25] and [26], for any chain extracted from a DAG application, the total completion time of the chain cannot exceeds the optimal makespan of the DAG application. Hence, we have that

$$\sum_{j=1}^p (T_{\min}(c_j) + T_c'(c_j)) \leq T_o. \quad (14)$$

Then, we can derive (11) based on (13) and (14). ■

Then, we consider the busy time in PLAYS.

Lemma 3: The total busy time of all servers in PLAYS must satisfy

$$\sum_{n \in \mathbb{N}} T_{\text{busy}}(n) \leq |\mathbb{N}| \frac{\beta_{\max}}{\beta_{\min}} T_o \quad (15)$$

where $T_{\text{busy}}(n)$ indicates the busy time of server n .

Proof: A server $n \in \mathbb{N}$ is regarded as busy whenever there is a container being started up or a layer being executed. Hence, the left-hand side of (15) is the time consumption

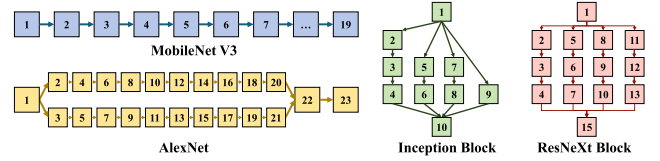


Fig. 4. Structure of DNN models.

of executing all scheduled DNN layers and starting up all corresponding containers. That is

$$\sum_{n \in \mathbb{N}} T_{\text{busy}}(n) = \sum_{n \in \mathbb{N}} (T_{\text{exe}}(n) + T_c(n)) \quad (16)$$

where $T_{\text{exe}}(n)$ the time of layer execution on server n . Obviously, $\sum_{n \in \mathbb{N}} T_{\text{exe}}(n)$ must be smaller than $(\beta_{\max}/\beta_{\min}) \sum_{l \in \mathbb{L}} T_{\min}(l)$. Hence, we have

$$\sum_{n \in \mathbb{N}} T_{\text{busy}}(n) \leq \sum_{l \in \mathbb{L}} \left(\frac{\beta_{\max}}{\beta_{\min}} T_{\min}(l) + T_c'(l) \right). \quad (17)$$

Following existing study [27], we have that $\sum_{l \in \mathbb{L}} (T_{\min}(l) + T_c'(l)) \leq |\mathbb{D}| T_o$, and hence (15) can be derived from (17). ■

With Lemmas 2 and 3, we can obtain the upper bound of T_p as follows.

Theorem 2: The inference time obtained PLAYS always satisfies

$$T_p \leq 2 \frac{\beta_{\max}}{\beta_{\min}} T_o + \sum_{p=1} T_{\max}^{\text{tran}}.$$

Proof: As T_p consists of the busy and idle time, it can be expressed as

$$T_p = \sum_{n \in \mathbb{N}} (T_{\text{idle}}(n) + T_{\text{busy}}(n)) / |\mathbb{N}|. \quad (18)$$

According to Lemmas 2 and 3, we have

$$T_p = \sum_{n \in \mathbb{N}} (T_{\text{idle}}(n) + T_{\text{busy}}(n)) / |\mathbb{N}| \leq 2 \frac{\beta_{\max}}{\beta_{\min}} T_o + \sum_{p=1} T_{\max}^{\text{tran}}. \quad \blacksquare$$

Therefore, our PLAYS algorithm can achieve inference time no worse than $2(\beta_{\max}/\beta_{\min})T_o + \sum_{p=1} T_{\max}^{\text{tran}}$.

V. EVALUATION AND ANALYSIS

In this section, to evaluate the effectiveness and efficiency of our PLAYS algorithm, we conduct extensive experiments and compare PLAYS with some state-of-art scheduling algorithms. We first introduce the default experiment settings.

A. Experiment Settings

1) *DNN Models:* Since we aim to assess the adaptability of PLAYS across a spectrum of DNN complexities and resource requirements, three specific DNN models are selected for evaluation: 1) MobileNet V3 [28]; 2) AlexNet [29]; 3) Inception V3 [30]; and 4) ResNeXt [31]. The MobileNet V3 is chosen for its extensive utilization and representative nature within the AIoT scenarios. In contrast, AlexNet, Inception V3 and ResNeXt models, with their distinct layer branching

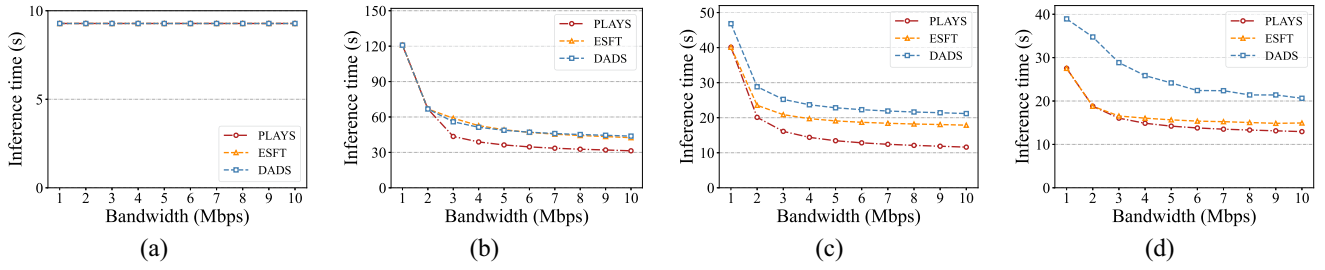


Fig. 5. Impact of varying average network bandwidth. (a) MobileNetV3. (b) AlexNet. (c) Inception V3. (d) ResNeXt.

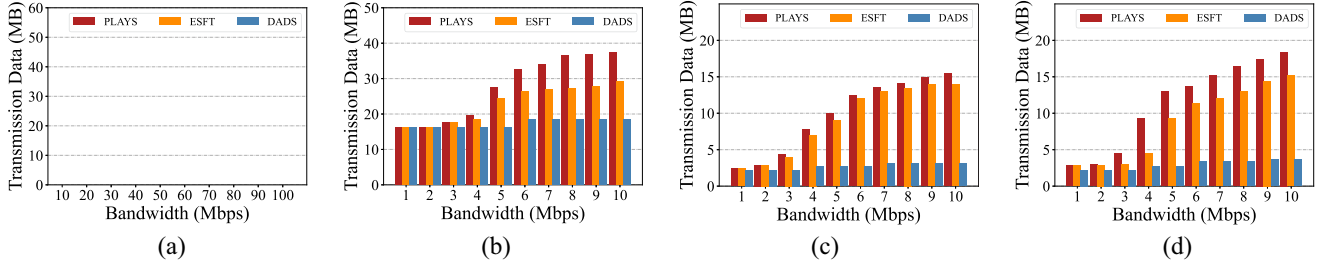


Fig. 6. Data volumes of intermediate results transmissions under varying average network bandwidth. (a) MobileNetV3. (b) AlexNet. (c) Inception V3. (d) ResNeXt.

structures, are chosen to simulate the impact of parallel network architectures on the algorithmic performance. The architectural details of these models are illustrated in Fig. 4. To emulate the inference workload accurately, floating-point operations serve as the basis for workload simulation in our experiments. This setting not only evaluates the performance of PLAYS but also sheds light on its efficacy in handling lightweight DNN models, which are prevalent in AIoT applications and align with the resource constraints typical in IoT environments. Additionally, the performance evaluation of other DNN models with comparable complexities and workloads can also be obtained by these selected representative models.

2) *Experiment Environment*: The simulations are conducted within a serverless edge cloud environment comprising 20 edge servers. The average computational resource capacity of edge servers is 10, and the computational power per computational resource can provide is 0.3GHz. Following the specifications provided in [32], the floating-point operations per CPU cycle are set at 0.7. By default, the average network bandwidth between any two edge servers is configured to be 50 Mb/s. These parameters collectively define the experimental setup, ensuring a consistent and realistic environment for evaluating the performance of PLAYS with various DNN models.

3) *Comparison Algorithms*: We compare our PLAYS against the following state-of-the-art algorithms.

- 1) *Earliest Start-Time First (ESTF) Algorithm* [33]: ESTF schedules the layers greedily in descending order of the earliest start time of the layers.
- 2) *Dynamic Adaptive DNN Surgery (DADS)* [34]: DADS splits the DNN model into two parts and executes them at end device and edge server, respectively.

B. Impact of Average Network Bandwidth

We first investigate the impact of average network bandwidth. In this experiment, we vary the average network bandwidth from 1MB to 10MB, and report the DNN inference time of the DNN models mentioned above under different average network bandwidths in Fig. 5. To better understand how the bandwidth affects the overall performance, we also report the intermediate data volumes transmitted between servers under different scheduling strategies in Fig. 6.

First of all, we notice that our PLAYS algorithm can always achieve the best performance under all DNN models and reduce the DNN inference time by 26.3% and 30% in comparison with ESFT and DADS, respectively. This proves that our PLAYS algorithm can effectively reduce the performance degradation brought by the container sequential creation and startup. However, as shown in Fig. 5, for MobileNet V3, we notice that ESFT and DADS can achieve comparable performance to PLAYS. We also observe that the volumes of intermediate transmission data generated by the three methods are all 0, i.e., no interserver transmission, as shown in Fig. 6(a). This is due to the fact that when meeting the DNN model with linear structure, the algorithms schedule all layers to one server for execution based on dependency relationships, resulting in no intermediate data transmission.

The results in Fig. 6 further illustrate the performance of the PLAYS from the perspective of intermediate transmission data volume under different network bandwidths. In situations where the network bandwidth is extremely constrained (i.e., 1–2 Mb/s), the time taken for data transmission becomes the dominant factor in the overall inference time. Consequently, all the algorithms try to minimize data transfer to avoid long intermediate data transmission time, resulting in low-data volume. With the increasing of bandwidth (i.e., 3–5 Mb/s), the data volume of both PLAYS and ESFT increase.

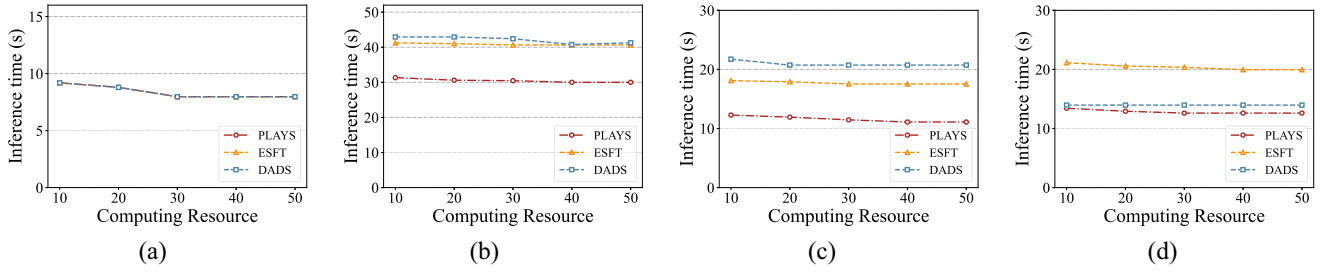


Fig. 7. Impact of varying numbers computing resource. (a) MobileNet V3. (b) AlexNet. (c) Inception V3. (d) ResNeXt.

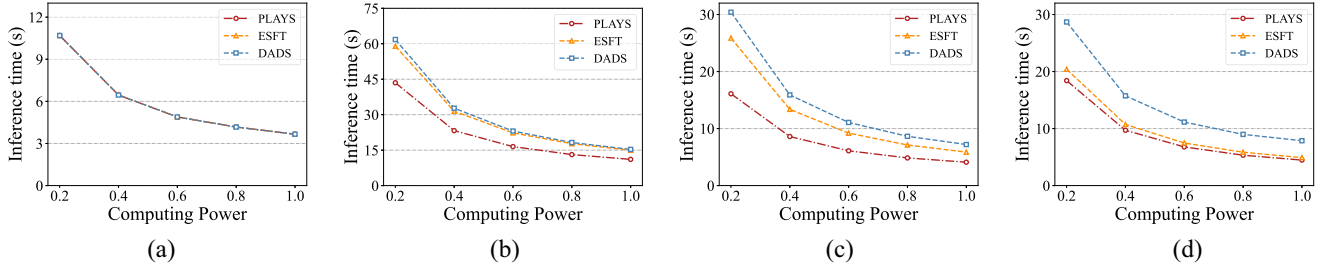


Fig. 8. Impact of varying server computing power. (a) MobileNet V3. (b) AlexNet. (c) Inception V3. (d) ResNeXt.

This is because higher increased bandwidth leads to lower transmission time, making it possible to schedule inference tasks to more distributed servers. As bandwidth continues to increase (i.e., 5–10 Mb/s), the influence of data transmission latency on the overall system performance is further reduced. Consequently, the container serial startup latency becomes the dominant aspect of the inference time. Our PLAYS is sensitive to such characteristics. Therefore, PLAYS has higher transmission data overhead than both ESFT and DADS, but it achieves the highest performance thanks to the well balance between transmission, startup, and execution.

C. Impact of Average Computational Resource Capacity

Besides the average network bandwidth, we further investigate the effect of the average computational resource capacity via varying the average computational resource capacity from 10 to 50. The experimental results are shown in Fig. 7.

Once again, due to the awareness of the container sequential creation characteristic, we can still notice that our PLAYS algorithm outperforms other state-of-art algorithms in all most cases. In comparison with ESFT and DADS, our PLAYS can speed up the inference performance by $1.33\times$ and $1.4\times$, respectively.

In addition, we find that the inference time of either DNN model decreases with the increase of average computational resource capacity. We attribute this phenomenon to that the higher computational resource capacity allows more functions to be scheduled to the powerful servers and hence reduces the layer execution time.

Furthermore, we notice that the inference time of MobileNet V3 remains constant when the average computational resource capacity exceeds 30MB. This is because the most powerful server has enough computational resource capacity to accommodate all the DNN layers for execution when the average computational resource capacity exceeds 30MB. In this case,

the average computational resource capacity is not the factor influencing the inference performance. Similar phenomenon can be also observed from Fig. 7(b) (i.e., when the average computational capacity exceeds 30) and Fig. 7(c) (i.e., when the average computational capacity exceeds 20).

D. Impact of Computational Power

Thanks to the heterogeneity of edge server, same computational resource may provide different computational power. Finally, we investigate how it influences the DNN inference performance and the adaptability of our algorithm. In this experiment, the average computational power per computational resource can provide (i.e., the value of β_n) is varied from 0.2GHz to 1.0GHz.

The experimental results are reported in Fig. 8, from which we observe that our proposed PLAYS can achieve the best performance, and reduce the inference time by 30.5% and 36.9% in comparison with ESFT and DADS, respectively. In addition, we notice that the inference time of all algorithms decreases with the increase of computational power. This can be attributed to that the higher computational power can effectively reduce the layer execution time, and hence imply lower inference time.

Another interesting finding is that, for AlexNet, PLAYS can only reduce the DNN inference time by 28%. However, when it comes to Inception V3 and ResNeXt, our PLAYS can reduce the DNN inference time by 40% and 38.2%. The main reason is that Inception V3 and ResNeXt have more complex structure than AlexNet. This means that there will be more containers created in parallel for executing the DNN layers. In this case, since the awareness of the container sequential creation characteristic, our PLAYS can reduce the overhead caused by waiting for container creation, and efficiently improve the inference performance.

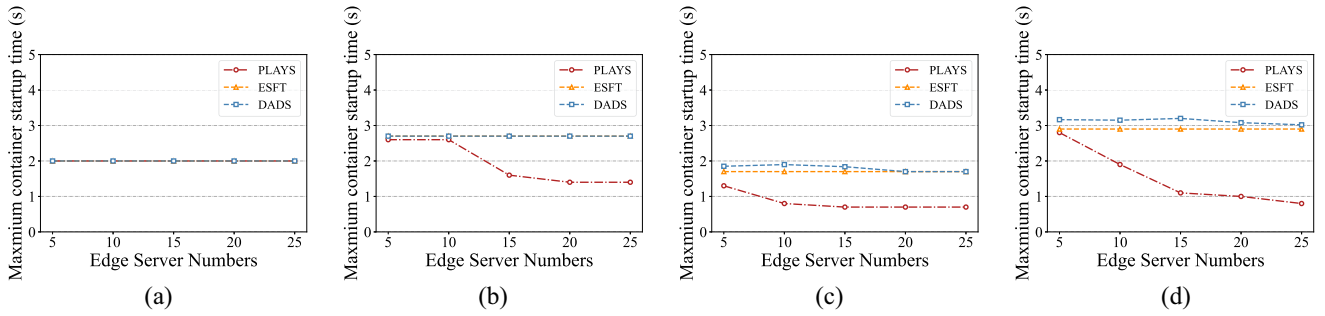


Fig. 9. Maximum container startup time under different numbers of edge servers. (a) MobileNet V3. (b) AlexNet. (c) Inception V3. (d) ResNeXt.

E. Elimination of Container Startup Time

Finally, we refocus our attention back to the essence of the inference latency in serverless edge cloud, which is the impact of container serial startup. To demonstrate the effectiveness of our proposed PLAYS in significantly eliminating the serial startup time of containers, we choose the maximum container startup time on a single server within the edge cloud as our metric. We vary the number of edge server from 5 to 25 to reflect the impact of startup time issues on serverless edge cluster.

Fig. 9 shows the maximum container startup time over all edge servers. In Fig. 9(a), we can see that all solutions show the same startup time, since MobileNet V3 is a purely sequential model and its layers must be started and executed one by one, leaving no optimization room. Meanwhile, the maximum container startup time of PLAYS is always shorter than both ESFT and DADS in Fig. 9(b)–(d). This is because when scheduling the layers, PLAYS also considers the startup time in *Priority(l)* and will find the edge server by balancing the computation resource (execution time) and IO bandwidth (startup time) at the same time. For example, with 20 edge servers, the maximum startup time of PLAYS is decrease averagely by 13.8% and 17% for AlexNet, 62.1% and 58.8% for Inception V3, and 69.4% and 70% for ResNeXt, compared with ESFT and DADS.

Furthermore, Fig. 9(b)–(d) show that the maximum startup time of ESFT and DADS remain essentially unchanged. This is because they only consider the computational power of the servers during task scheduling and ignore the effects of startup time. It can also be observed that the maximum startup time of PLAYS decreases with more edge servers involved in Fig. 9(b)–(d), indicates the optimization potential of our PLAYS in large scale edge platforms.

VI. CONCLUSION

In this article, we investigate how to accelerate DNN-based AIoT application performance in serverless edge cloud. With the consideration of container sequential creation characteristic, we formally formulate the optimization problem aiming at minimizing DNN inference time into a nonlinear integer. Due to the NP-hardness of this problem, we further propose an approximation algorithm called PLAYS, and theoretically analyze its performance upper bound. To evaluate the effectiveness and adaptability of our proposed PLAYS, we conduct extensive trace-driven simulation experiments to

compare PLAYS with some state-of-the-art algorithms under different DNN models. The experimental results show that our PLAYS outperforms other existing algorithms, and significantly reduces the inference time by 28%–40% averagely, showcasing its potential as a robust solution for accelerating DNN-based AIoT applications at the serverless edge. Moving forward, we anticipate that PLAYS will serve as a foundational framework for future research, inspiring further advancements in the domain of serverless edge computing and the broader field of AIoT.

REFERENCES

- [1] C.-T. Yang, H.-W. Chen, E.-J. Chang, E. Kristiani, K. L. P. Nguyen, and J.-S. Chang, "Current advances and future challenges of AIoT applications in particulate matters (PM) monitoring and control," *J. Hazardous Mater.*, vol. 419, Oct. 2021, Art. no. 126442.
- [2] W. Ullah et al., "Artificial intelligence of things-assisted two-stream neural network for anomaly detection in surveillance big video data," *Future Gener. Comput. Syst.*, vol. 129, pp. 286–297, Apr. 2022.
- [3] H. Lin, G. Liu, F. Li, and Y. Zuo, "Where to go? Predicting next location in IoT environment," *Front. Comput. Sci.*, vol. 15, pp. 1–13, Feb. 2021.
- [4] M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX Conf. Usenix Annu. Tech.*, 2020, pp. 205–218.
- [5] J. Shao and J. Zhang, "Communication-computation trade-off in resource-constrained edge inference," *IEEE Commun. Mag.*, vol. 58, no. 12, pp. 20–26, Dec. 2020.
- [6] T. Rausch, A. Rashed, and S. Dustdar, "Optimized container scheduling for data-intensive serverless edge computing," *Future Gener. Comput. Syst.*, vol. 114, pp. 259–271, Jan. 2021.
- [7] Y. Yang et al., "INFless: A native serverless system for low-latency, high-throughput inference," in *Proc. 27th ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2022, pp. 768–781.
- [8] L. Feng, P. Kudva, D. Da Silva, and J. Hu, "Exploring serverless computing for neural network training," in *Proc. IEEE 11th Int. Conf. cloud Comput. (CLOUD)*, 2018, pp. 334–341.
- [9] M. Yu, Z. Jiang, H. C. Ng, W. Wang, R. Chen, and B. Li, "Gillis: Serving large neural networks in serverless functions with automatic model partitioning," in *Proc. IEEE 41st Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 2021, pp. 138–148.
- [10] A. Mohan, H. Sane, K. Doshi, S. Edupuganti, N. Nayak, and V. Sukhomlinov, "Agile cold starts for scalable serverless," in *Proc. 11th USENIX Workshop Hot Topics Cloud Comput. (HotCloud 19)*, 2019, pp. 1–6.
- [11] E. Oakes et al., "SOCK: Rapid task provisioning with serverless-optimized containers," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2018, pp. 57–70.
- [12] L. Zeng, E. Li, Z. Zhou, and X. Chen, "Boomerang: On-demand cooperative deep neural network inference for edge intelligence on the industrial Internet of Things," *IEEE Netw.*, vol. 33, no. 5, pp. 96–103, Sep./Oct. 2019.
- [13] X. Chen, M. Li, H. Zhong, Y. Ma, and C.-H. Hsu, "DNNOff: Offloading DNN-based intelligent IoT applications in mobile edge computing," *IEEE Trans. Ind. Informat.*, vol. 18, no. 4, pp. 2820–2829, Apr. 2022.

- [14] T. Mohammed, C. Joe-Wong, R. Babbar, and M. Di Francesco, "Distributed inference acceleration with adaptive DNN partitioning and offloading," in *Proc. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, 2020, pp. 854–863.
- [15] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, "CoEdge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, Apr. 2021.
- [16] L. Wu, R. Zhang, Q. Li, C. Ma, and X. Shi, "A mobile edge computing-based applications execution framework for Internet of Vehicles," *Front. Comput. Sci.*, vol. 16, no. 5, 2022, Art. no. 165506.
- [17] A. Ali, R. Pincirol, F. Yan, and E. Smiri, "BATCH: Machine learning inference serving on serverless platforms with adaptive batching," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2020, pp. 1–15.
- [18] J. Cho, D. Z. Tootaghaj, L. Cao, and P. Sharma, "SLA-driven ML inference framework for clouds with heterogeneous accelerators," in *Proc. Mach. Learn. Syst.*, 2022, pp. 20–32.
- [19] S. Wang, Z. Ding, and C. Jiang, "Elastic scheduling for microservice applications in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 98–115, Jan. 2021.
- [20] H. Dai, J. Wu, Y. Wang, and C. Xu, "Towards scalable and efficient deep-RL in edge computing: A game-based partition approach," *J. Parallel Distrib. Comput.*, vol. 168, pp. 108–119, Oct. 2022.
- [21] Y. Duan and J. Wu, "Computation offloading scheduling for deep neural network inference in mobile computing," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Service (IWQOS)*, 2021, pp. 1–10.
- [22] N. Wang, Y. Duan, and J. Wu, "Accelerate cooperative deep inference via layer-wise processing schedule optimization," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2021, pp. 1–9.
- [23] X. Gong, L. Guo, G. Shen, and G. Tian, "Virtual network embedding for collaborative edge computing in optical-wireless networks," *J. Lightw. Technol.*, vol. 35, no. 18, pp. 3980–3990, Sep. 2017.
- [24] J. Lou, H. Luo, Z. Tang, W. Jia, and W. Zhao, "Efficient container assignment and layer sequencing in edge computing," *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1118–1131, Mar./Apr. 2023.
- [25] E. Bampis and A. V. Kononov, "Bicriteria approximation algorithms for scheduling problems with communications delays," *J. Sched.*, vol. 8, no. 4, pp. 281–294, 2005.
- [26] D. R. Karger, C. Stein, and J. Wein, "Scheduling algorithms," in *Algorithms and Theory of Computation Handbook* (Applied Algorithms and Data Structures Series). Boca Raton, FL, USA: Chapman Hall/CRC, 1999.
- [27] J.-J. Hwang, Y.-C. Chow, F. D. Anger, and C.-Y. Lee, "Scheduling precedence graphs in systems with interprocessor communication times," *SIAM J. Comput.*, vol. 18, no. 2, pp. 244–257, 1989.
- [28] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1314–1324.
- [29] C. Szegedy et al., "Intriguing properties of neural networks," 2014, *arXiv:1312.6199*.
- [30] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 2818–2826.
- [31] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He, "Aggregated residual transformations for deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1492–1500.
- [32] "V. Nikolskiy and V. Stegailov, "The performance of floating-point operations in ARM cores and their efficiency in classical molecular dynamics," *J. Phys., Conf. Ser.*, vol. 681, no. 1, 2016, Art. no. 12049.
- [33] M. Y. Özkaya, A. Benoit, B. Uçar, J. Herrmann, and Ü. V. Çatalyürek, "A scalable clustering-based task scheduler for homogeneous processors using DAG partitioning," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2019, pp. 155–165.
- [34] C. Hu, W. Bao, D. Wang, and F. Liu, "Dynamic adaptive DNN surgery for inference acceleration on the edge," in *Proc. IEEE Int. Conf. Comput. Commun. (INFOCOM)*, 2019, pp. 1423–1431.



Hongmin Geng received the B.S. and M.S. degrees from the School of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing, China, in 2016 and 2020, respectively. He is currently pursuing the Ph.D. degree in geographic information system with China University of Geosciences, Wuhan, China.

His current research interests mainly focus on edge intelligence, serverless computing, and related technologies like task scheduling and AIoT.



Deze Zeng (Senior Member, IEEE) received the B.S. degree from the School of Computer Science and Technology, Huazhong University of Science and Technology, China in 2007, and the M.S. and Ph.D. degrees in computer science from University of Aizu, Aizu-Wakamatsu, Japan, in 2009 and 2013, respectively.

He is currently a Full Professor with the School of Computer Science, China University of Geosciences, Wuhan, China. His current research interests mainly focus on edge computing, and related technologies like network function virtualization, machine learning, and IoT. He has authored three books and over 120 papers in refereed journals and conferences in these areas.

Dr. Zeng also received three Best Paper Awards from IEEE/ACM conferences and the IEEE Systems Journal Annual Best Paper Award of 2017 and 2019. He serves in editorial boards for IEEE TRANSACTIONS ON SUSTAINABLE COMPUTING, *Journal of Network and Computer Applications*, *Frontiers of Computer Science*, and *Open Journal of Computer Society*, and the Guest Editor of many prestigious journals/magazines. He has been the in organization or program committees of many international conferences. He is a Senior Member of CCF.



Yuepeng Li received the B.S. and M.S. degrees from the School of Computer Science, China University of Geosciences, Wuhan, China, in 2016 and 2019, respectively, and the Ph.D. degree in geographic information system from China University of Geosciences in 2023.

His current research interests mainly focus on edge computing, and related technologies like task scheduling, trusted execution environment, and IoT.



Lin Gu (Member, IEEE) received the M.S. and Ph.D. degrees in computer science from The University of Aizu, Fukushima, Japan, in 2011 and 2015, respectively.

She is currently a Associate Professor with the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China. Her current research interests include cloud native, network function virtualization, and edge computing. She has authored two books and more than 40 papers in refereed journals and conferences in these areas.

Dr. Gu is a Senior Member of CCF.



Quan Chen (Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China, in June 2014.

He is currently a Tenure-Track Associate Professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include high-performance computing, task scheduling in various architectures, resource management in datacenter, runtime system, and operating system.



Peng Li (Senior Member, IEEE) received the B.S. degree from School of Computer Science and Technology, Huazhong University of Science and Technology, China, in 2007, and the M.S. and Ph.D. degrees in computer science from University of Aizu, Aizu-Wakamatsu, Japan, in 2009 and 2012, respectively.

He is currently an Associate Professor with The University of Aizu, Aizuwakamatsu, Japan. He has published more than 100 technical papers on prestigious journals and conferences. His research interests mainly focus on cloud/edge computing, Internet of Things, machine learning systems, as well as related wired and wireless networking problems.

Dr. Li won the Young Author Award of IEEE Computer Society Japan Chapter, in 2014. He won the Best Paper Award of IEEE TrustCom 2016. He supervised students to win the First Prize of IEEE ComSoc Student Competition in 2016. He is the Editor of *IEICE Transactions on Communications* and *IEEE OPEN JOURNAL OF THE COMPUTER SOCIETY*.